Spark
POLYTECHNIC

قسم
هندسة الطاقة
الكهربائية
فريق سبارك

ملخص:

لاب أسمبلي

Exp 1: Machine Code and Z3/EV Experimental kit

• Part 1: Machine code:

← احنا بنعرف إنو الكمبيوتر بفهم بلغة الـ (Machine Code) عشان هيك لغة الاسمبلي هي فقط لتسهيل التعامل بين الانسان والكمبيوتر.

فمثلاً لو عندنا الكود التالي :

MOV AL, 2FH ← لغة الاسمبلي

1011 0 000 10 1111 ← Machine code (لغة الآلة)

- 1011 → MOV تمثل
- 0 → تمثل نوع السجل byte
- 000 → اسم السجل AL=000 BL=011
- 10 1111 → القيمة المخزنة 2FH وهنا

← أي كود بلغة الاسمبلي بقدر احوله للغة الآلة عن طريق الترتيب التالي :

[Code | w | Reg | Imm.Data]

- Code ← الأمر فمثلاً MOV ≡ 1011 ... وهكذا.
- w ← نوع السجل: w=0 → byte 8bit مثل AL BL CL DL ; w=1 → word 16bit مثل AX BX CX DX
- Reg ← اسم السجل مثلاً AL=000 BL=011 CL=001 DL=010
- Imm.Data ← الرقم المراد تخزينه في السجل او القيمة

← مثال: ماهو الكود المكافئ بلغة الآلة لـ MOV AL, 0F2H

الحل:

1011, 0, 000, 1111 0010
MOV, 8bit, AL, 0F2H

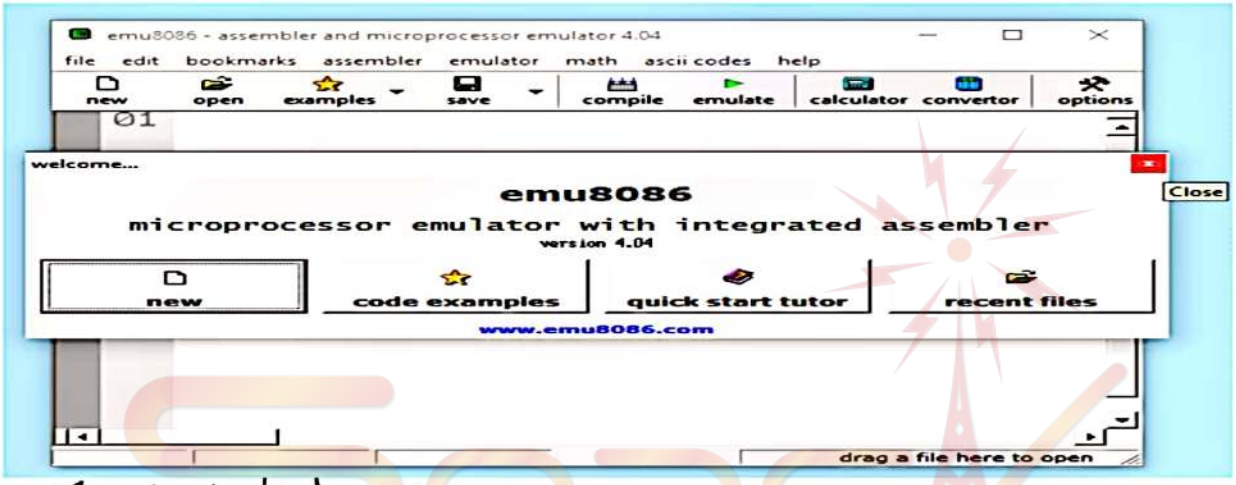
ملاحظة: تم وضع صفر قبل الـ F عشان الـ assymbler ما يحسب الـ H كحرف.

• Part 2 : Emu 8086 :

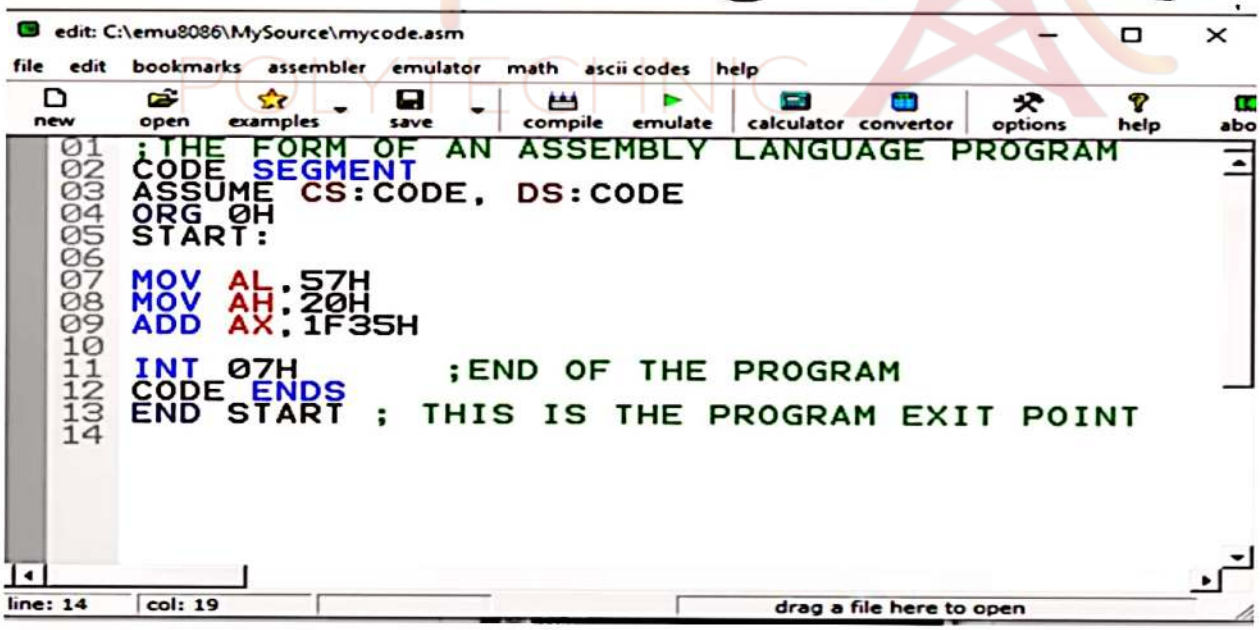
← البرنامج اللي راح نشتغل عليه بالمختبر هو [Emu8086] هو برنامج خفيف ومسهل للتعامل للمبتدئين. يحتوي على محرر للأكواد وحاسبة ومحول.

← نتعرف على واجهة البرنامج شوي :

1. بشغل البرنامج الموجود على سطح المكتب أو قائمة ابدأ Emu8086.exe .
2. بغلق شاشة الترحيب .

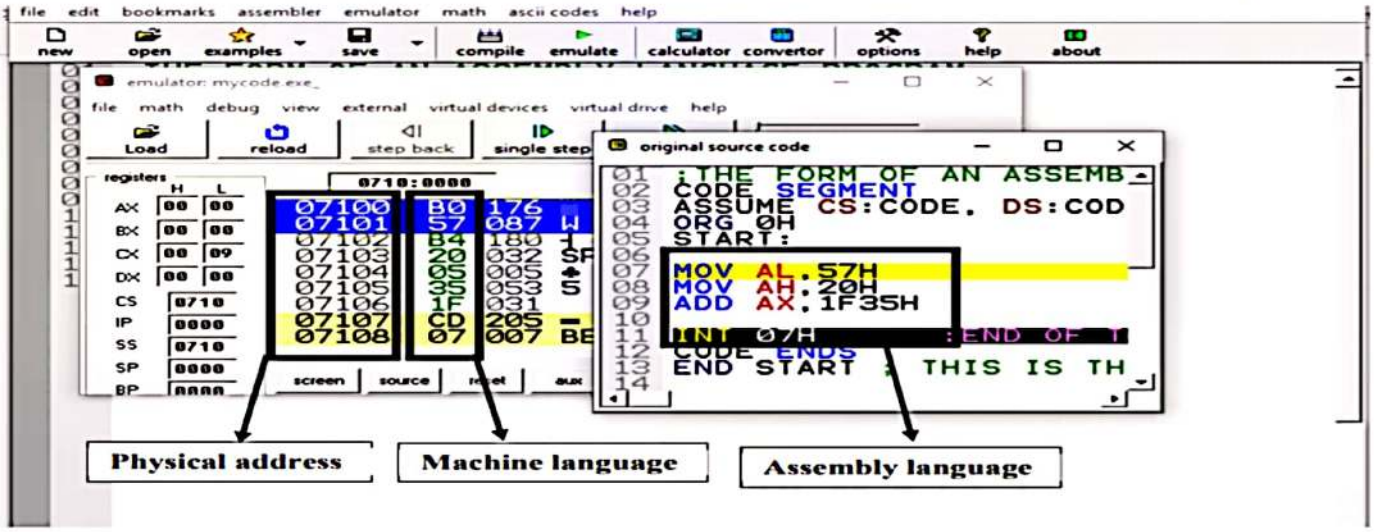


3. بدخل الكود على البرنامج مع مراعاة المسافات والفواصل والأحرف .



4. بعمل emulate فقط ما بعمل Run للبرنامج .

5 راح يطلعلك شاشة فيها كل المعلومات اللي بدك إياها .



- Part 3: Logical address and physical address :

Logical address

(CS : IP)

16bit 16bit

0710 : 0000

0710 : 0001

0710 : 0010

⋮

البرفسور بيقلك انك تقبلش منه هاد بس بالدرج بتقبلش منه

0000 : 0800

physical address

كيف احسب الـ physical address من الـ Logical address ؟

بشكل بسيط :

1. بعمل إزاحة لليسار لـ [CS] .
2. بضيف [IP] لـ [CS] .

مثال : Find physical address for

0710:0000

CS : IP

الحل:

1. إزاحة لليسار لـ 0710 بصير 07100
2. بضيف 0000 لـ 07100

الناتج 07100 هو الـ Physical address

Part 4: Z3/EV Experimental Kit :



أهم الإستعمالات الموجودة داخل الـ Kit :

1. Microprocessor 386EX at 221 MHz.
2. 32KB EPROM system, 32KB EPROM users 32KB RAM static.
3. keyboard hexadecimal / commands.
4. Display LCD with a line of 16 characters.
5. Parallel interface (8+8+4 lines I/o), Serial interface RS-232.
6. analogue inputs and outputs 0-8V convoter of 8bit
7. logic probes
8. signale supply 5V.
9. monitor in EPROM with commands for: visualization and modification memory and registers, and continuous execution.

كيف اعمل Run على الـ Kit ؟

① لبشغل برنامج Emu8086 على الكمبيوتر

② بدخل الكود المطلوب

③ بعمل Emulate

④ بأخذ الـ machine code

⑤ بروح على الـ Kit وبكتب على لوحة المفاتيح LD_KB

⑥ بحدد الموقع الابتدائي اللي هو 0000:0800

⑦ بدخل الـ machine code عن طريق اللوحة الموجودة على الـ Kit

⑧ بعمل Run.

Exp 2: keyboard interfacing with 8086 MP using Z3/EV Exp. kit

* **interrupt:** is a condition that halts the microprocessor temprority to work on a different task and then return to its previous task.

← حالة يتوقف فيها المعالج مؤقتاً لعمل وظيفة معينة ثم يعود للوظيفة السابقة.

← الـ kit اللي بنستخدمو بالمختبر بدعم 20 نوع من الـ interrupt كل واحد منهم بعمل وظيفة معينة بهاي التجربة راح نشرح [INT 08H].

→ **INT 08H:**

→ used for reading of a key from the keyboard

← ليستخدم لقراءة رقم من الكيبورد وتخزينه في المسجل [AL].

- input: —
- output: AL = key code pressed (0-18)
- Altered Reg.: —

← مثال: راح ادخل INT 08H راح يستنى من البرنامج اكبس على رقم من الكيبورد (الشغل على الـ kit) الرقم اللي كبستو بتخزن بالمسجل AL

بعدين بعمل MOV BL,AL صارت القيمة مخزنة بالـ BL.

* Note: JNZ

[Jump if Zero flag not Zero]: اختصار

Ex:

```
MOV BL,05H   → عداد 5 مرات
Next:
=========
Dec BL       → كل مرة بنقص 1 من العداد
JNZ Next
```

JNZ Next
- ① Zf = 0
- ② BL = 0

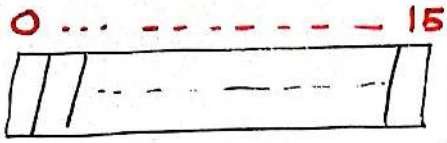
* عندي شرطين عشان اطلع من اللوب

Interruption Number	Routine Name	General Description
INT 00H	IMONITOR	Division by 0. Return to the monitor.
INT 01H	reserved	Single-step
INT 02H	Not used	Interruption non mask
INT 03H	Reserved	Breakpoints
INT 04H	Not used	Overflow
INT 05H	Not used	
INT 06H	Not used	
INT 07H	IMONITOR	End user prog. and return to the monitor
INT 08H	IKEYBOARD	Reading of a key from the keyboard
INT 09H	IDIS_BYTE	Sending of hexadec. byte to the display
INT 0AH	IDIS_CHAR	Sending of ASCII character to the display
INT 0BH	IDIS_OUTS	Sending of a string to the display
INT 0CH	IDIS_CODE	Sending commands to the display
INT 0DH	IWAIT_MS	Wait in milliseconds
INT 0EH	IAD_READ	Reading from A/D converter
INT 0FH	IDA_WRITE	D/A converter Command
INT 10H	IBUZZER	Buzzer Command
INT 11H	Not used	
INT 12H	IPARAL	Management parallel interface
INT 13H	Not used	
INT 14H	ISERIAL	Management serial interface

انواع ال interrupt

Exp3: LCD controlling using Z3/Ev Experimental kit

ملاحظة: نستخدم (INT 07H) لإنهاء البرنامج.



(1X16) أبعاد الشاشة

[1] INT 09H: → Sending of a byte in hexadecimal form to the display.
← ارسال بايت بصيغة الهيكسا إلى الشاشة

- input: ① CL = position on the display (0-14).
← اطبع بال AL بدلاً على موقع المؤشر على الشاشة القيم المسموحة من (0-14)
ليش مصفر لـ 15؟ لأنو رقم الهيكسا مكون من خانتين بدهم مالية وجود كان رقم خانة 15.
② AL = byte to display.
← القيمة المخزن في السجل AL.
- output: —
- altered Reg: —

[2] INT 0AH: → Sending of ASCII character to the display.
← ارسال رمز ASCII للشاشة. تستخدم هذه التقاطعة لإظهار حرف على الشاشة.

- input: ① CL = position on the display (0-14).
② AL = ASCII code of the character.
- output: —
- altered Reg: —

[3] INT 0BH: → sending of string of character on the display.
← ارسال مجموعة أرقام أو أحرف للشاشة.

- input: ① DS: indecates the segment containing the string.
② SI: indecates the begining address of the string.

① * ملاحظة: ما بقدر أخزن في DS مباشرة فلازم أعمل نقل لسجل آخر ثم إلى DS

مثل:

```
MOV AX, 0080H
MOV DS, AX
```

دايماً بعملها لما استخدم INT 0BH.

```
MOV DS, 0080H
```

* ملاحظة: في الامر INT 0BH - يجب ان تنتهي الجملة بـ 00H .

Ex: MSG DB '1+1=2', 00H

4 INT 0C H: → sending commands to the display.
← ارسال اوامر الى الشاشة .

- input: AH=1, clear the display.
  AH=2, Bring the cursor home.
  AH=3, Moves Cursor right.
  AH=4, Moves Cursor Left.
  AH=5, Cursor On.
  AH=6, Cursor off.
  AH=7, Bring Cursor in position (AL).
- output: —
- altered Reg: —

5 INT 0DH: → wait in milli second
← اعطاء مهلة زمنية للأمر لكي نستطيع رؤيته على الشاشة .

- input: AX = number of milli second
- output: —
- altered Reg: —

* مثال:

```
MOV AX,1000
INT 0DH
```

* هاي التعليمة بتعطي delay ثانية .
* كثير راح نستخدمها .

Exp 4: string Operations using Z3/Ev Exp. kit

The table below provides a summary of all the string instructions.

Instruction	Mnemonic	Destination	Source	Prefix
move string byte	MOVSB	ES:DI	DS:SI	REP
move string word	MOVSW	ES:DI	DS:SI	REP
store string byte	STOSB	ES:DI	AL	
store string word	STOSW	ES:DI	AX	REP
load string byte	LODSB	AL	DS:SI	none
load string word	LODSW	AX	DS:SI	none
compare string byte	CMPSB	ES:DI	DS:SI	REPE/REPNE
compare string word	CMPSW	ES:DI	DS:SI	REPE/REPNE
scan string byte	SCASB	ES:DI	AL	REPE/REPNE
scan string word	SCASW	ES:DI	AX	REPE/REPNE

- Program1: Program to interchange two blocks of data

```
        Mov AX,0080h
        Mov DS,AX
        Mov CX, 0004h; Initialize cx reg with count
        Mov BX, 0000h  ; store bx with 00
Repeat:
        Mov AL, src[BX]  ; move the contents at location o to al
        Xchg AL, dst[BX]  ; Excahnge the contents of al with dst
        Mov src [BX], al  ; copy the contents of al to src
        Inc BX  ; Increment bx by 1
        Loop Repeat  ; go to the label repeat if cx != 0
        Int 07h
        src db 31h,32h,33h,34h
        dst db 41h,42h,43h,44h
```

- Program2: Program to check whether given number is Odd or Even

```
        Mov AX,0080h
        Mov DS, AX
        Int 08h ;read key
        Ror AL,1    ; rotate right to check theLSB
        JC odd    ; check if CF = 1, then the number is odd
        ; else even
        Mov SI, Offset   msg1  ; Code to display a message
        Int 0bh
        Jmp exit
Odd:    Mov SI, Offset   msg2  ; code to display a message
        Int 0bh
Exit:
        Mov AX,2000
        Int 0dh
        Int 07h    ; Terminate the program
        msg1 db 'Even',00h
        msg2 db 'Odd' ,00h
```

- Program3: Program to move a string from source to destination

```
Mov AX,0080h
Mov DS, AX
Mov ES, AX
Mov CX,16
Lea SI,src
Lea DI,dst
Rep Movsb
Mov SI, Offset   dst
Int 0bh
Mov AX,2000
Int 0dh
Int 07h
src db 'Micro' ,00h
dst db 00h
```

- Program4: Moving Block of String

```
Mov AX,0080h
Mov DS, AX
Mov ES, AX
Mov  SI, Offset   STR1
Mov  DI, Offset   STR2
Mov   CX, 0AH
CLD
Rep Movsb
Mov SI, Offset   STR2
Int 0bh
Mov AX,2000
Int 0dh
Int 07h
STR1    DB   'HIFRIEND', 00h
STR2    DB    00h
```

Spark POLYTECHNIC

Exp5: parallel I/O unit (part I): LEDS.

← برايه التجربة راح نتعرف على انواع الـ ports و كيف اخلي الـ Leds يشتغلوا بالشكل اللي بدي اياه .

→ First: Ports :

← في عندي 3 ports رئيسيات هم :

Port A
Port B
Port C

- عاشان يشتغلوا كـ input او output حسب الـ mode تاع 8255 .

[1] port A: [A0-A7]

15-22 من Pins عال يستخدم ← . Connector الـ

input ← يقدر استخدمو ← output

[2] Port B: [B0-B7]

7-14 من Pins عال يستخدم ←

input ← يقدر استخدمو ← output

[3] Port C:

→ [C0-C3]

Pins عال يستخدم ←

5 - 3 - 23 - 25

. connector الـ

→ [C4]

Buzzer تعليمات الـ ←

[راح نوقف تجربة كاملة الا].

output كـ يستخدم فقط ←

← الـ modes تاعت 8255 هم :

1. mode 0 ← بكل الدروب راح نشتغل عهاد بس
2. mode 1
3. mode 2

← شو يعني mode 0 !؟

يعني الـ Ports A+B يقدر اي مجموعة كـ input or output

اما portC ببرمجو بشكل منفصل عنهم

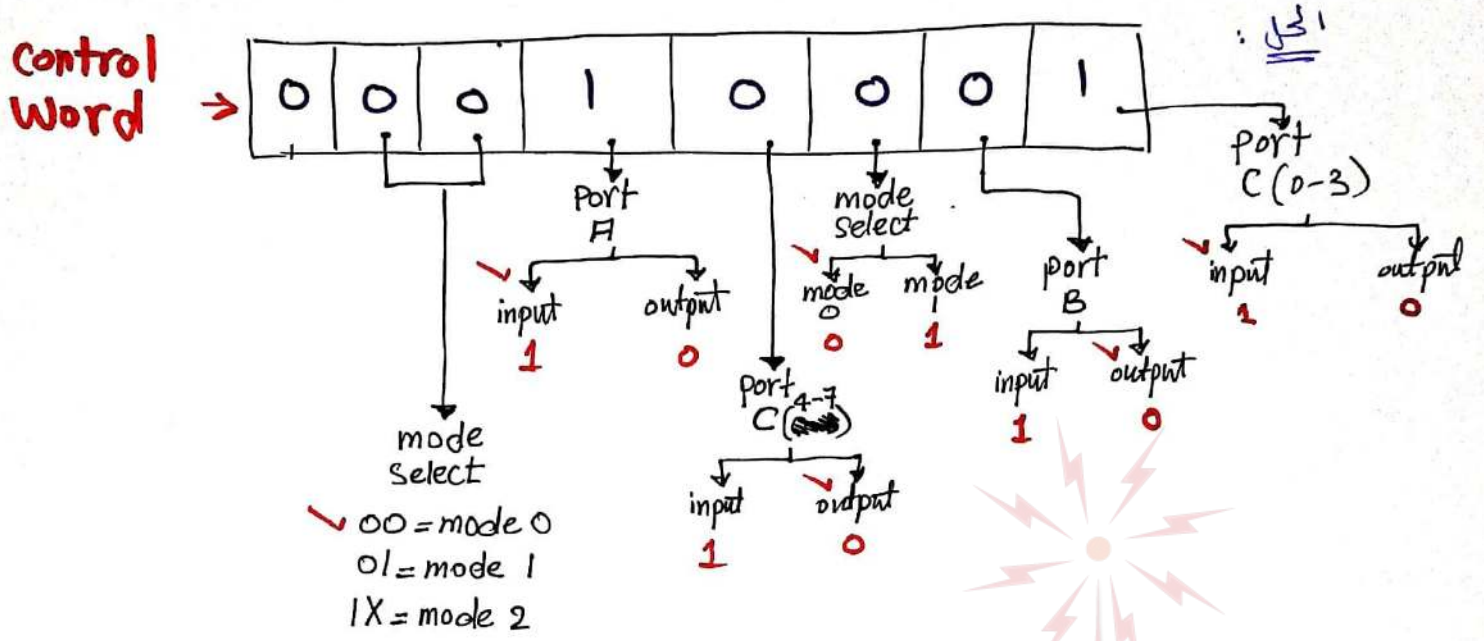
SPARK POLYTECHNIC

* Control word :

طيب ؟! كيف راح ابرمج 8255 ؟
الجواب عن طريق (control word)

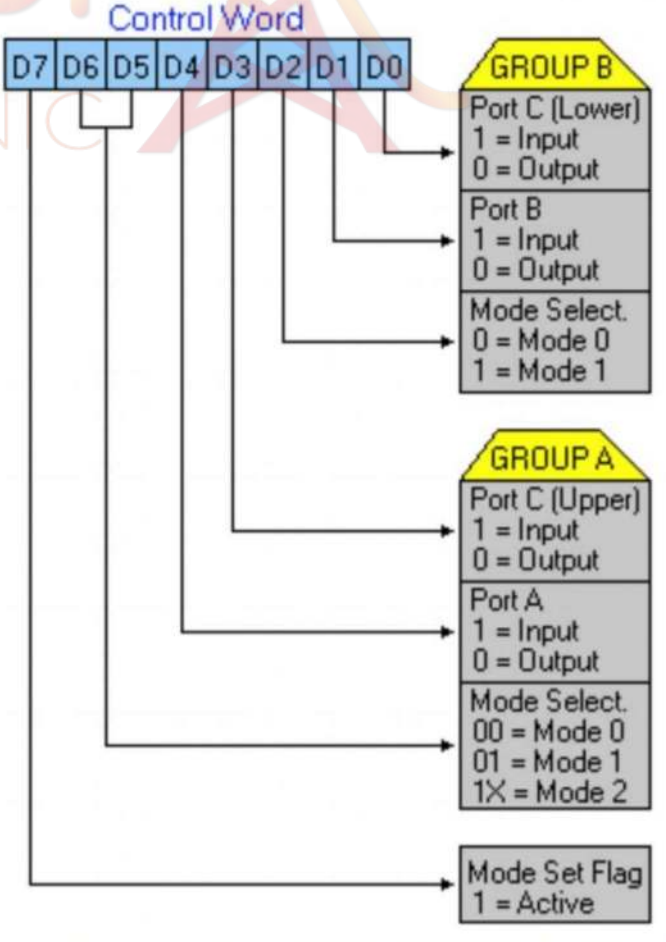
مثال : حابيتلك بدي Port A input / Port B output / Port C input (0-3) / Port C output (4-7)
شو نعمل ؟

الحل :



عشان اطبق هاد الكلام عالبرنامج بكتب :

```
MOV AL, 00010001B
MOV DX, 0313H
OUT DX, AL
```



• INT 12 H ← هسا بدنا نشوف كيف اشغل الـ Leds عن طريق

* حكي شوي عن (INT 12H) بعدين نوخذ مثال نفهم كلشي :

INT 12 H: Managment of the parallel Interface.

← يستخدم لإدارة الواجهة المتوازية .

• input: AH = 0 → Ports برمجة الـ.

AH = 1, 2, 3 → Sending data to ports A, B, C.

AH = 4, 5, 6 → Reading data to ports A, B, C.

• output:

AL = data read → (الشكل اللي بدي اطلعو)

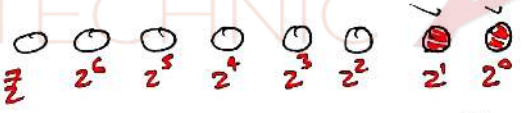
Ex: Write an Assembly program to produce the following patterns.

*Note: port A output
port B output
port C output



Sol:

```
MOV AH, 00H  →  Ports عشان اعمل برمجة الـ
MOV AL, 00H  →  حسب الـ Control word
INT 12H      →  هسا البرنامج قرأ AH, AL و خزن إنو الـ Ports output
MOV BL, 03H  →  BL بشوف الشكل اللي عندي و خزن قيمتو بـ
```



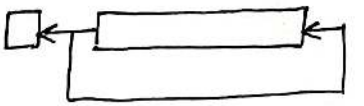
الشكل قيمتو 03H

```
MOV AH, 01H  →  عشان هيك port A بدي ابعت الـ data لـ
                AH=1 خليت .
MOV AL, BL   →  AL بجيب الـ data وخبأ بـ
                لأنو هو الـ output اللي
INT 12H
MOV AX, 1000 ]
INT 0DH      ] → delay ← عشان يبين الضوء
INT 07H      →  نهيت البرنامج
```

هسا آخر اشي - أخيراً - لو بدي اعمل نقط باد Leds شو اعمل ؟!

- راح نستخدم ROL , ROR أخذناهم بالمادة .

* ROL:



* ROR:



- طرق عمل ROR , ROL :

1. ROL AX, 1
2. MOV CL, 04H
   ROL BL, CL
3. ROL factor[BX], 1

} → أنا بفضل هاي ^^

- طريقة الحل :

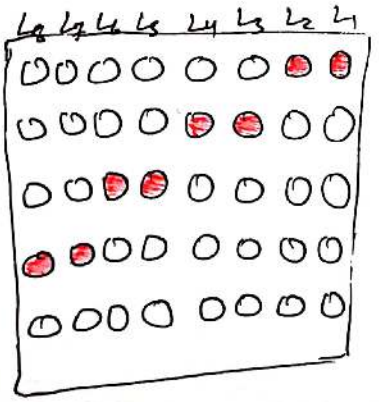
MOV AX, 00H
INT 12H
L_1:
MOV BL, [ ] → أول سطر من النقط
MOV CL, [ ] → كم مرة تكرر النقط أو كم سطر النقط
L_2:
MOV AH, 01H → عشان ارسل لـ Port A
MOV AL, BL → نبعت الـ data لـ AL
INT 12H
ROR لليمين / ROL لليسار
BL, [ ] → كم مرة راح انزح النقط
Dec CL
JNZ L_1
JMP L_2

Ports وجبة AH=0 / Control word حسب AL=20

* ملاحظة :

هاد البرنامج بستخدمو فقط لما بيكون النقط بكل الاسطر متشابه ، ولكن مزاج لليمين أو اليسار .

مثال :



* حط delay عشان يظهر النقط عالشاشة .

Exp 6: parallel I/O unit (part II): BUZZER.

← تجربة سهلة وبسيطة راح نتعلم كيف نتحكم بالجرس الموجود على الـ Kit.

← راح نستخدم INT 10H مخصص للجرس :

INT 10 H: use of the Buzzer.

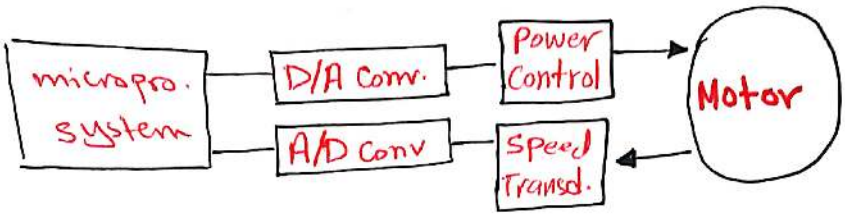
- input: BX = duration. مدة التشغيل
  CX = frequency. تردد الجرس
- output: —

← عشان اشغل الجرس بكتب :

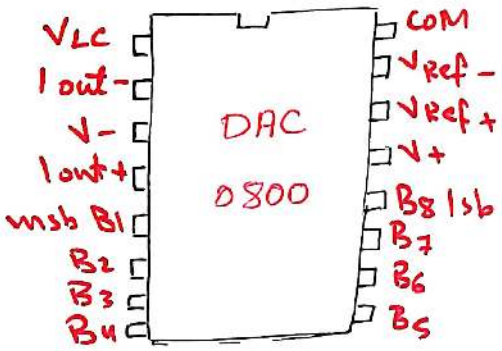
```
MOV AH,00H   } تجهيز الـ ports
MOV AL,00H   } وتحديد هل هم input
INT 12H      } ولا output

MOV BX,200   → مدة الـ Beep
MOV CX,20    → تردد الـ Beep
INT 10H      → أول ما أنبس انتر بيطلع صوت Beep
INT 07H      → بنهي البرنامج
```

Exp 7: Digital/Analog Conversion



• Converter DAC 0800 :→



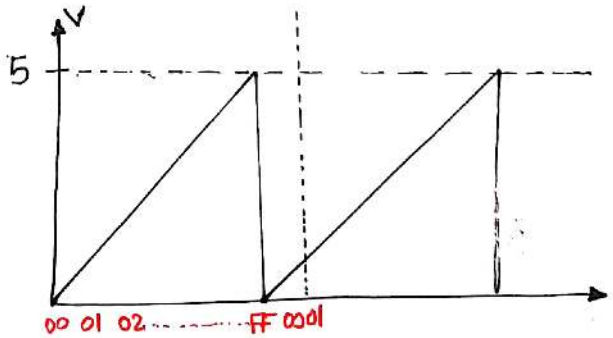
* V_{LC} = 0 Volt for the TTL.
* $V_{Ref}-$: connected to 0V for the unipolar functioning.
* B1-B8 : Digital inputs.

• INT 0FH : Command of the DAC.

يرسل الداتا إلى DAC الذي يحولها تلقائياً إلى analogue value.

- input : AL = data.
- output: —

Ex: develop a program that uses the D/A converter to generate waveform with saw teeth shape.



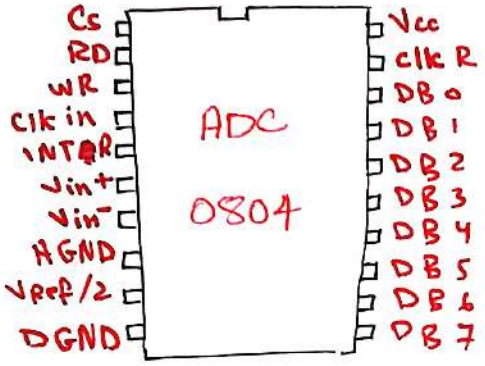
Sol:

```
START:
   MOV AL, 0FFH
TLOOP:
   INC AL
   INT 0FH  → out to D/A
   JMP TLOOP
```

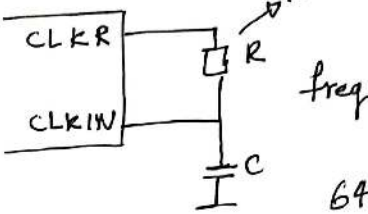
⇒ Resolution of the D/A converter = $\frac{\text{Full scale Voltage}}{2^N}$ → input is N-bit number

Exp 8: Analog/Digital Conversion

- Converter **ADC 0804** :-



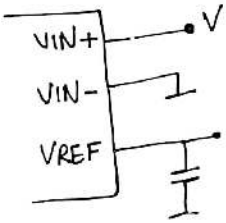
→ clock :



$$freq. = \frac{1}{1.1 * R * C}$$

* بالعادة التردد يكون 640KHz

→ analog inputs :



- INT 0EH : Reading of the ADC ⟶
  - input : —
  - output : AL = result of the conversion.

① يرسل امر بدء التحويل من انالوج الى ديجيتال.
② ينتظر اشارة نهاية التحويل ويقرأ نتيجة التحويل ويعيدها إلى AL.

Ex: develop a program that reads from the ADC and visualizes the result in hexadecimal form on the display.

* هذه مثال عملية :

Sol :

```
MOV AX,0080h
MOV DS,AX
MOV SI,offset MSG
INT 0BH
TLOOP:
INT 0EH
MOV CL,13
INT 09H
MOV AX,200
INT 0DH
JMP TLOOP
MSG DB 'A/D value: XX', 00H
```

1. Digital output various from 0 to 255
2. when $V_{ref} = 5V$, for every 19.53 mV of analog value there will be rise of one bit on digital size (step size).
3. Analog voltage = Decimal value * step size.
4. $\text{step size} = \frac{V_{ref}}{2^{\text{Resolution of ADC}}}$